

Optimizare neliniară

Cuprins

Problema 1	1
Modelarea problemei	1
Variabile	1
Funcția obiectiv de minimizat	1
Restricții	1
Caracterizarea problemei	1
Optimizare matematică	1
Optimizare statică	1
Optimizare parametrică	2
Optimizare cu restricții	2
Programare continuă	2
Programare în numere reale	2
Programare neliniară	2
Programare convexă	2
Programare deterministă	2
Programare uniobiectiv	2
Problemă de optimizare convexă	2
Calcularea derivatelor de ordinul 1	3
Calcularea derivatelor de ordinul 2	4
Verificarea matricei Hesse	6
Concluzia	7
Graficul funcției obiectiv	7
Graficul restricțiilor	8
Reprezentarea funcției obiectiv pe domeniul de fezabilitate	9
Dimensiunea spațiului de căutare	9
Numărul de soluții valide (fezabile)	9
Ordinul de mărime al soluțiilor valide	9
Modelarea problemei folosind PySCIPOpt	9

Schimbarea instanțelor.....	10
Tabel de restricții.....	11
Tabel cu rezultatele	11
Observații.....	11
Implementarea metodei Newton cu repornire.....	11
Implementarea metodei Simulated Annealing	13
Analiza comparativă a rezultatelor și a metodelor folosite	15
Comparația performanțelor pe metode	15
Concluzii.....	15

Problema 1

La o rafinărie se urmărește să se producă un amestec de două produse petroliere pentru care nivelul de emisii nocive la utilizare să fie minim. Temperaturile celor două produse în momentul amestecării (t_1 și t_2) determină nivelul de emisii nocive al amestecului după formula $\ln(2^{t_1} + 2^{t_2})$. Având în vedere că primul produs se poate adăuga la amestec la o temperatură între -0.4°C și 2°C , iar al doilea între 3.2°C și 10°C și că temperaturile celor două împreună trebuie să depășească 4.5°C , cum trebuie procedat pentru a obține amestecul cu nivel minim de emisii nocive?

Modelarea problemei

Variable

T_1 – temperatura primului produs

T_2 – temperatura celui de al doilea produs

Funcția obiectiv de minimizat

$$\min f(t_1, t_2) = \ln(2^{t_1} + 2^{t_2})$$

Restricții

$$(R1) \ t_1 \geq -0.4 \text{ } ^\circ\text{C}$$

$$(R2) \ t_1 \leq 2 \text{ } ^\circ\text{C}$$

$$(R3) \ t_2 \geq 3.2 \text{ } ^\circ\text{C}$$

$$(R4) \ t_2 \leq 10 \text{ } ^\circ\text{C}$$

$$(R5) \ t_1 + t_2 \geq 4.5 \text{ } ^\circ\text{C}$$

Caracterizarea problemei

Optimizare matematică

Este optimizare matematică, deoarece se cunoaște explicit funcția obiectiv și restricțiile, nu se bazează pe simulări sau teste empirice.

Optimizare statică

Este optimizare statică, întrucât datele de intrare nu se modifică de la începerea rezolvării problemei până la găsirea soluției.

Optimizare parametrică

Este optimizare parametrică, deoarece variabilele (t_1 , t_2) sunt scalari, deci funcția obiectiv are parametri scalari, nu parametri funcții.

Optimizare cu restricții

Este optimizare cu restricții, existând 5 restricții explicite.

Programare continuă

Este programare continuă, întrucât toate variabilele de decizie (necunoscuta compusă) trebuie să primească valori continue (pot primi o infinitate de valori) – grade de temperatură ce pot fi numere reale.

Programare în numere reale

Este o programare în numere reale, neavând nicio restricție care să ne indice faptul că variabilele t_1 și t_2 trebuie să fie numere întregi, acestea reprezentând temperaturile a două produse pot lua valori din mulțimea numerelor reale.

Programare neliniară

Este programare neliniară, pentru că funcția obiectiv este neliniară.

Programare convexă

Problema este o problemă de optimizare convexă, deoarece în urma aplicării condiției de ordinul 2 al funcțiilor convexe s-a demonstrat folosind Hessianul funcției că elementele de pe diagonala principală sunt ≥ 0 și că determinantul este ≥ 0 (în cazul de față egal cu 0), ceea ce înseamnă că funcția f este convexă.

Programare deterministă

Este programare deterministă întrucât toate datele de intrare (temperaturile celor 2 produse) sunt cunoscute apriori și nu intervin variabile aleatoare.

Programare uniobiectiv

Este programare uniobiectiv, deoarece există un singur obiectiv urmărit: minimizarea nivelului de emisii nocive. Nu se cere optimizarea simultană a mai multor obiective.

Problemă de optimizare convexă

O funcție este **convexă** dacă Hessianul său (matricea de derivate parțiale ale acesteia) este **pozitiv semidefinit** și **concavă** dacă Hessianul este **negativ semidefinit**.

Calcularea derivatelor de ordinul 1

$$(a^x)' = a^x \ln a$$

$$f(t_1, t_2) = \ln(2^{t_1 + 2t_2})$$

$$(\ln u)' = \frac{1}{u} \cdot u'$$

Calculăm derivatele de ordinul 1

$$\frac{df}{dt_1} = \frac{d}{dt_1} \ln(2^{t_1 + 2t_2}) \quad u = 2^{t_1 + 2t_2}$$

$$= \frac{d}{du} \ln u \cdot \frac{d}{dt_1} (2^{t_1 + 2t_2}) =$$

$$= \frac{1}{2^{t_1 + 2t_2}} \cdot 2^{t_1} \cdot \ln 2 = \frac{2^{t_1} \cdot \ln 2}{2^{t_1 + 2t_2}}$$

$$\frac{df}{dt_2} = \frac{d}{dt_2} \ln(2^{t_1 + 2t_2}) \quad u = 2^{t_1 + 2t_2}$$

$$= \frac{d}{du} \ln u \cdot \frac{d}{dt_2} (2^{t_1 + 2t_2}) =$$

$$= \frac{1}{2^{t_1 + 2t_2}} \cdot 2^{t_2} \cdot \ln 2 = \frac{2^{t_2} \cdot \ln 2}{2^{t_1 + 2t_2}}$$

Calcularea derivatelor de ordinul 2

$$\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$$

Calculăm derivata de ordinul 2

$$\begin{aligned} \frac{\partial^2 f}{\partial t_1^2} &= \frac{\partial}{\partial t_1} \left(\frac{\partial f}{\partial t_1} \right) = \frac{\partial}{\partial t_1} \frac{z^{t_1} \cdot \ln z}{z^{t_1} + z^{t_2}} = \\ &= \frac{(z^{t_1} \ln z)' \cdot (z^{t_1} + z^{t_2}) - z^{t_1} \ln z \cdot (z^{t_1} + z^{t_2})'}{(z^{t_1} + z^{t_2})^2} = \\ &= \frac{z^{t_1} \cdot \ln z \cdot \ln z \cdot (z^{t_1} + z^{t_2}) - z^{t_1} \cdot \ln z \cdot z^{t_1} \cdot \ln z}{(z^{t_1} + z^{t_2})^2} = \\ &= \frac{\ln z^2 \cdot z^{t_1} \cdot (z^{t_1} + z^{t_2}) - z^{2t_1} \cdot \ln z^2}{(z^{t_1} + z^{t_2})^2} = \\ &= \frac{\ln z^2 \cdot z^{2t_1} + \ln z^2 \cdot z^{t_1+t_2} - z^{2t_1} \cdot \ln z^2}{(z^{t_1} + z^{t_2})^2} = \\ &= \frac{\ln z^2 \cdot z^{t_1+t_2}}{(z^{t_1} + z^{t_2})^2} \end{aligned}$$

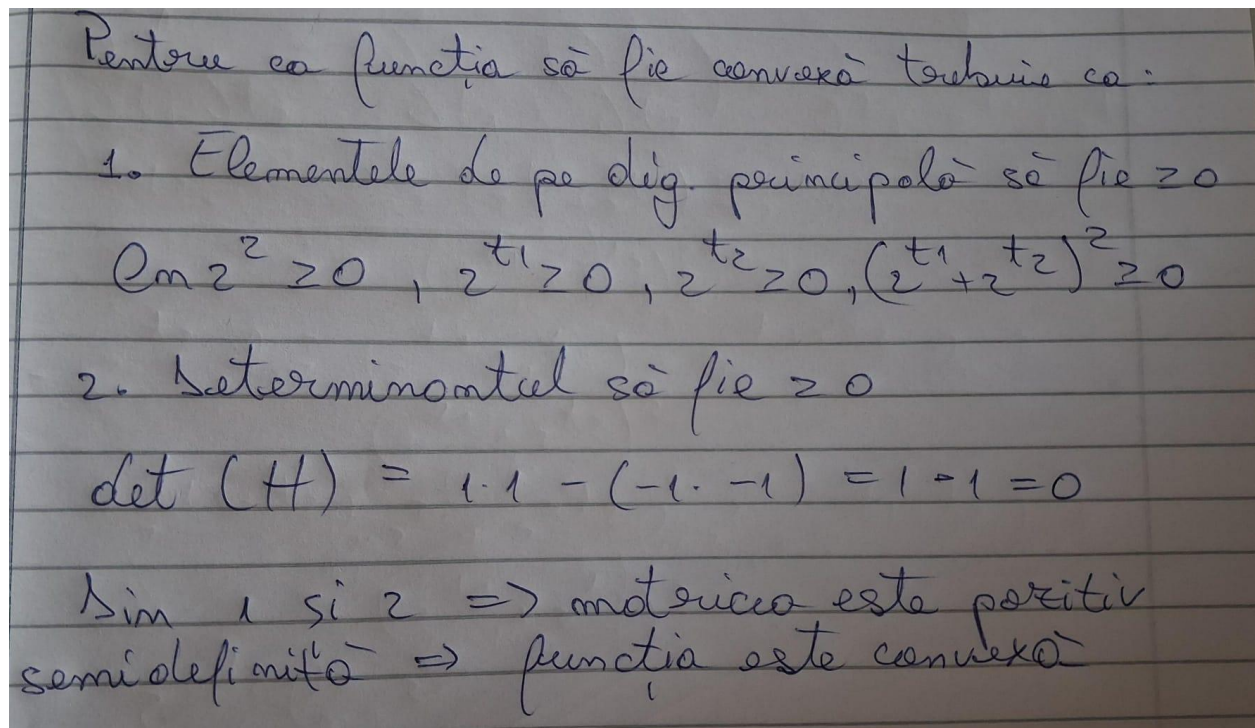
$$\begin{aligned}
\frac{\partial^2 f}{\partial t_2^2} &= \frac{\partial}{\partial t_2} \left(\frac{\partial f}{\partial t_2} \right) = \frac{\partial}{\partial t_2} \frac{z^{t_2} \cdot \ln z}{z^{t_1} + z^{t_2}} = \\
&= \frac{(z^{t_2} \cdot \ln z)' (z^{t_1} + z^{t_2}) - z^{t_2} \ln z \cdot (z^{t_1} + z^{t_2})'}{(z^{t_1} + z^{t_2})^2} = \\
&= \frac{z^{t_2} \cdot \ln z \cdot \ln z \cdot (z^{t_1} + z^{t_2}) - z^{t_2} \ln z \cdot z^{t_2} \cdot \ln z}{(z^{t_1} + z^{t_2})^2} = \\
&= \frac{z^{t_2} \ln z^2 \cdot (z^{t_1} + z^{t_2}) - z^{2t_2} \cdot \ln z^2}{(z^{t_1} + z^{t_2})^2} = \\
&= \frac{z^{t_1+t_2} \cdot \ln z^2 + \cancel{z^{2t_2} \cdot \ln z^2} - \cancel{z^{2t_2} \ln z^2}}{(z^{t_1} + z^{t_2})^2} = \\
&= \frac{\ln z^2 \cdot z^{t_1+t_2}}{(z^{t_1} + z^{t_2})^2}
\end{aligned}$$

$$\begin{aligned}
 \frac{\partial^2 f}{\partial t_1 \partial t_2} &= \frac{\partial}{\partial t_1} \left(\frac{\partial f}{\partial t_2} \right) = \frac{\partial}{\partial t_1} \cdot \frac{2^{t_2} \cdot \ln 2}{2^{t_1} + 2^{t_2}} = \\
 &= \frac{(2^{t_2} \cdot \ln 2)' \cdot (2^{t_1} + 2^{t_2}) - 2^{t_2} \cdot \ln 2 (2^{t_1} + 2^{t_2})'}{(2^{t_1} + 2^{t_2})^2} = \\
 &= \frac{0 \cdot (2^{t_1} + 2^{t_2}) - 2^{t_2} \ln 2 \cdot 2^{t_1} \ln 2}{(2^{t_1} + 2^{t_2})^2} = \\
 &= \frac{-\ln 2^2 \cdot 2^{t_1+t_2}}{(2^{t_1} + 2^{t_2})^2} = -\frac{\ln 2^2 \cdot 2^{t_1+t_2}}{(2^{t_1} + 2^{t_2})^2}
 \end{aligned}$$

Verificarea matricei Hesse

Verificarea matricei Hesse

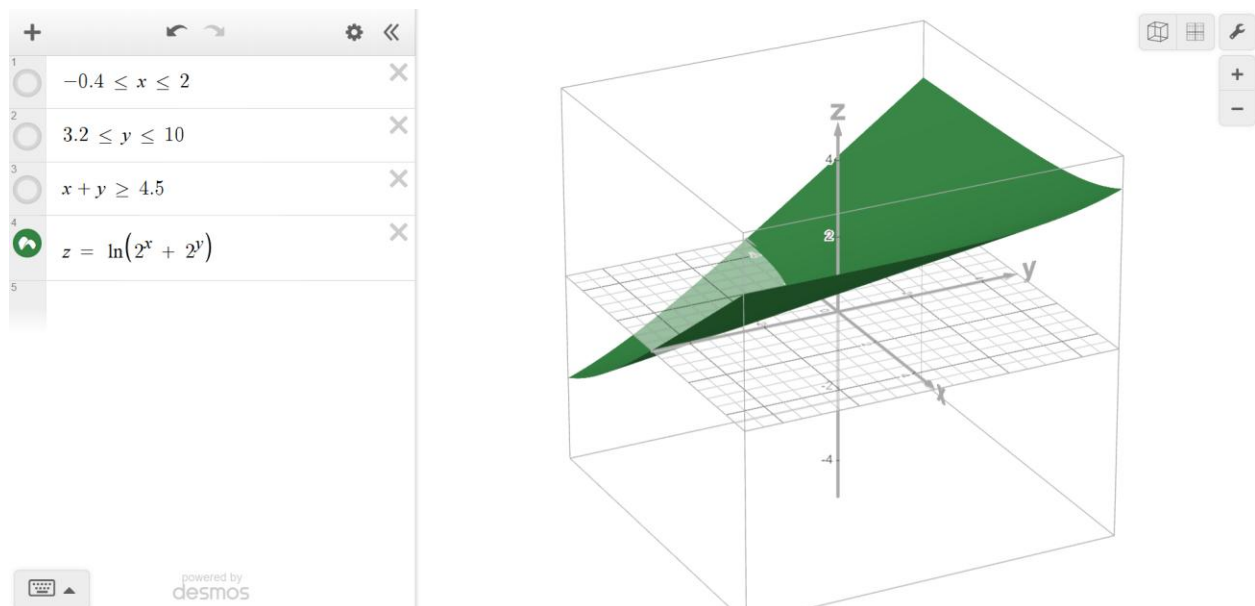
$$\begin{aligned}
 H &= \begin{pmatrix} \frac{\ln 2^2 \cdot 2^{t_1+t_2}}{(2^{t_1} + 2^{t_2})^2} & -\frac{\ln 2^2 \cdot 2^{t_1+t_2}}{(2^{t_1} + 2^{t_2})^2} \\ -\frac{\ln 2^2 \cdot 2^{t_1+t_2}}{(2^{t_1} + 2^{t_2})^2} & \frac{\ln 2^2 \cdot 2^{t_1+t_2}}{(2^{t_1} + 2^{t_2})^2} \end{pmatrix} = \\
 &= \frac{\ln 2^2 \cdot 2^{t_1+t_2}}{(2^{t_1} + 2^{t_2})^2} \begin{pmatrix} 1 & -1 \\ 1 & -1 \end{pmatrix}
 \end{aligned}$$



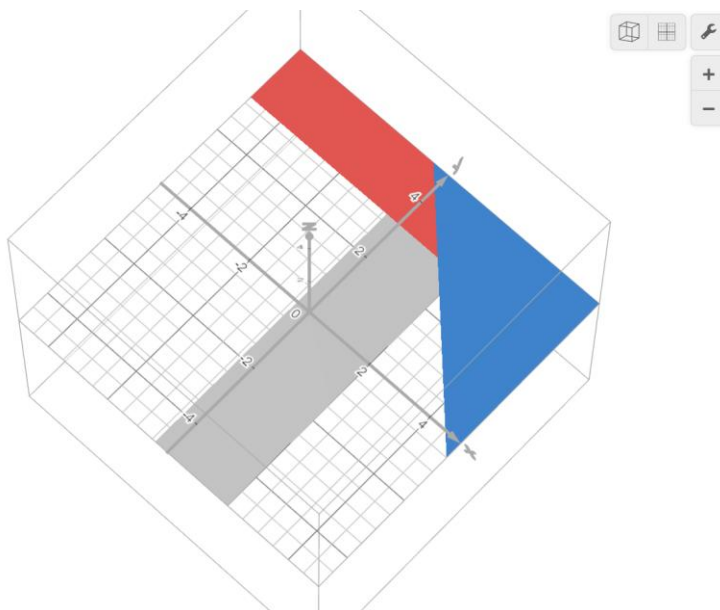
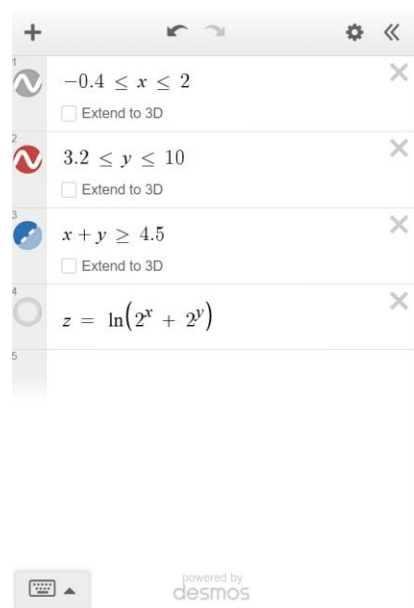
Concluzia

Problema este o problemă de optimizare convexă, deoarece în urma aplicării condiției de ordinul 2 al funcțiilor convexe s-a demonstrat folosind Hessianul funcției că elementele de pe diagonala principală sunt ≥ 0 și că determinantul este ≥ 0 (în cazul de față egal cu 0), ceea ce înseamnă că funcția f este convexă.

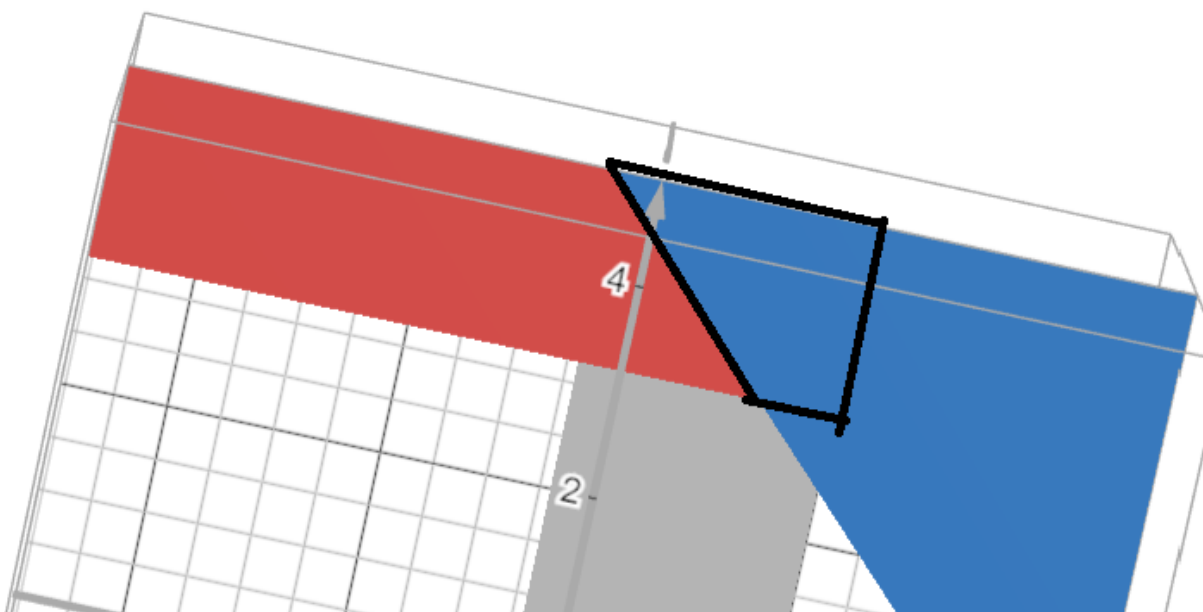
Graficul funcției obiectiv



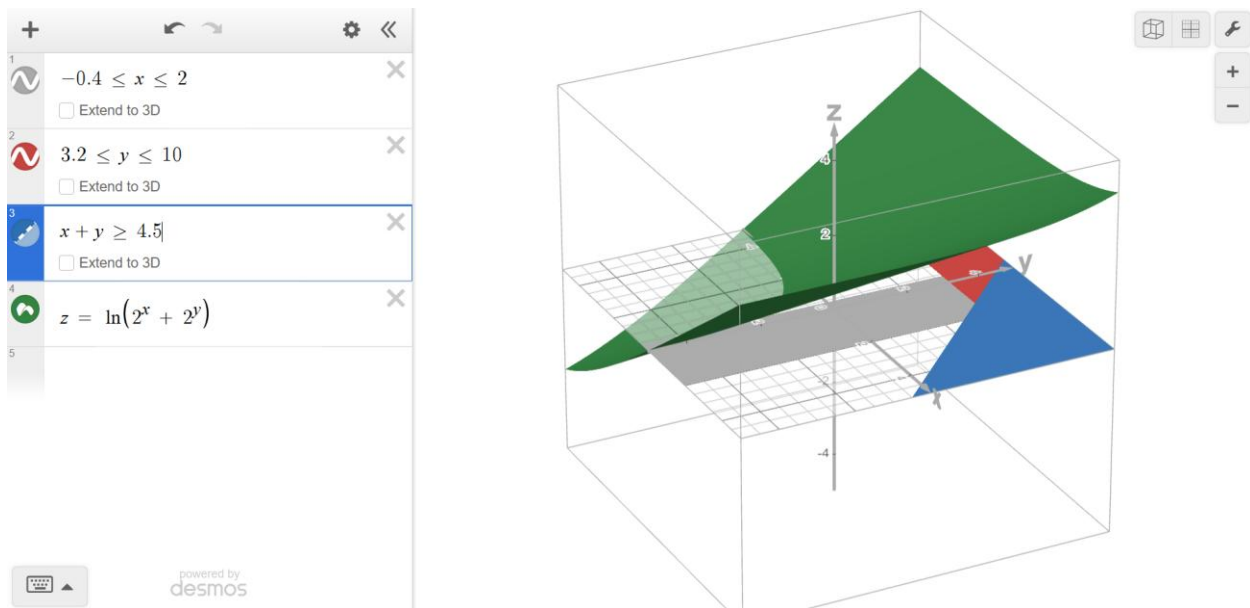
Graficul restricțiilor



Din grafic se poate observa că mulțimea soluțiilor fezabile este intersecția celor 3 restricții, așa cum se poate observa și din figura de mai jos.



Reprezentarea funcției obiectiv pe domeniul de fezabilitate



Link către grafic - <https://www.desmos.com/3d/has9vwcdnj>

Dimensiunea spațiului de căutare

Dimensiunea spațiului de căutare este 2, deoarece avem 2 variabile de decizie independente (t_1 și t_2).

Numărul de soluții valide (fezabile)

Deoarece problema este continuă, adică variabilele pot lua orice valoare reală în intervalele lor ($[-0.4, 2]$ și $[3.2, 10]$), numărul de soluții valide este infinit.

Ordinul de mărime al soluțiilor valide

Pentru a determina cât de mare e acest spațiu, trebuie să determinăm aria în care putem căuta valori. Dacă nu ținem cont de restricția $t_1 + t_2 \geq 4.5$ atunci:

- Lungimea intervalului pentru t_1 va fi $2 - (-0.4) = 2.4$ unități
- Lungimea intervalului pentru t_2 va fi $10 - 3.2 = 6.8$ unități

Aria va fi egală cu 2.4×6.8 adică 16.32 unități pătrate. **$A = 2.4 \times 6.8 = 16.32$**

În urma aplicării restricției spațiul se va micșora.

Modelarea problemei folosind PySCIPOpt

```
from pycipopt import Model
from pycipopt import Model, log, exp
```

```

model = Model(problemName="Problema1") # numele modelului (Problema1)

x = model.addVar("t1", lb=-0.4, ub=2, vtype="C") # continua intre -0.4 si 2
y = model.addVar("t2", lb=3.2, ub=10, vtype="C") # continua intre 3.2 si 10
obj = model.addVar("obj", lb=None, vtype="C") # variabila auxiliara pentru
obiectiv

# nu putem folosi operatorul clasic python ** pentru ridicarea la putere
# ne vom folosi de relatia 2^x = e^x*ln2 pentru a face o schimbare de baza
# functia exp(x) calculeaza valoarea lui e la puterea x
# ln(2^x + 2^y) devine ln(exp(x * ln2) + exp(y * ln2))
ln2 = 0.69314718056

model.addCons(obj >= log(exp(x * ln2) + exp(y * ln2))) # adaugam o
restrictie neliniara
model.setObjective(obj, "minimize") # minimizarea variabilei auxiliare care
este liniara

model.addCons(x + y >= 4.5, name="prag_minim") # restrica sumei
temperaturilor

model.optimize()

status = model.getStatus()
print("Status:", status)

if status == "optimal":
    x_val = model.getVal(x)
    y_val = model.getVal(y)
    print("x =", x_val)
    print("y =", y_val)

    import math
    rezultat = math.log(2**x_val + 2**y_val)

    print("ln(2^x + 2^y) = ", rezultat)
else:
    print("Nu s-a gasit o solutie optima.")

```

Datele obținute în urma apelării solver-ului s-au scris în tabelul de mai jos. Pentru a se obține valoarea emisiilor minimă cele 2 temperaturi trebuie să fie cât mai mici, astfel pentru t1 s-a obținut valoarea 1,2(9), iar pentru t2 s-a obținut 3,1(9). Adunate aceste numere reale ne dau un număr ce nu depășește 4,5, dar care este extrem de apropiat de el și anume 4,99999996. Aceste valori ne dau un nivel de emisii de 2,455 ce este considerat nivelul cel mai scăzut pe care îl putem avea.

t1 °C	t2 °C	t1 + t2 °C	ln(2 ^{t1} + 2 ^{t2})
1.2999999864727871	3.199999977219736	4.499999996	2.4554671540482187

Schimbarea instanțelor

În schimbarea instanțelor s-a urmărit în mare parte schimbarea temperaturii minime a celor 2 produse și schimbarea temperaturii minime acceptate a produselor combinate.

Datele modificate vor fi prezentate în primul tabel de mai jos (prima linie fiind datele inițiale), iar rezultate obținute se pot vedea în al doilea tabel de mai jos (primul rând fiind cel al rezultatelor obținute fără să se fi făcut vreo modificare asupra setului de restricții). Pentru restricții s-au păstrat exact aceleași semne, doar valorile au fost modificate.

X – este valoare care se modifică

Tabel de restricții

Setul de date	$t1 \geq x \text{ }^{\circ}\text{C}$	$t1 \leq x \text{ }^{\circ}\text{C}$	$t2 \geq x \text{ }^{\circ}\text{C}$	$t2 \leq x \text{ }^{\circ}\text{C}$	$t1 + t2 \leq x \text{ }^{\circ}\text{C}$
1	-0.4	2	3.2	10	4.5
2	-0.4	2	3.5	10	4.5
3	1.2	2	3.5	10	4.5
4	1.2	2	3.5	10	4.8
5	1.2	2	3.5	10	5.1

Tabel cu rezultatele

Sdd	$t1 \text{ }^{\circ}\text{C}$	$t2 \text{ }^{\circ}\text{C}$	$t1 + t2 \text{ }^{\circ}\text{C}$	$\ln(2^{t1} + 2^{t2})$
1	1.2999999864727871	3.199999977219736	4.49999996	2.4554671540482187
2	0.9999999901530618	3.499999965177352	4.49999995	2.588794197079301
3	1.2	3.5	4.7	2.610886019022423
4	1.2999999895115455	3.499999965639455	4.79999995	2.6229277326966773
5	1.5999998988925923	3.500000101107407	5.09999999	2.663411363113474
Media emisiilor nocive				2.58824

Observații

În urma datelor adunate în tabelele de mai sus se poate observa că atunci când temperaturile minime ale celor 2 produse nu dau adunate exact temperatura minimă necesară pentru îndeplinirea restricției numărul 5, valorile temperaturilor obținute sunt numere reale ce adunate doar cu un număr finit de zecimale vor da o valoare foarte apropiată de cea necesară pentru îndeplinirea ultimei restricții, totuși, mai mică.

În urma analizei făcute putem observa cum mereu produsul 2 a avut o temperatură minimă, și a fost crescută doar temperatura primului produs pentru a se ajunge la respectarea restricției 5.

Implementarea metodei Newton cu repornire

S-a ales implementarea metodei Newton, deoarece converge de regulă la un optim mai rapid decât metoda gradientului. În plus, metoda ne indică dacă s-a ajuns la un maxim global sau local, nu la un minim sau un punct de inflexiune.

Funcția noastră este o funcție convexă și derivabilă de două ori, metoda Newton fiind special creată pentru așa ceva, folosindu-se de a doua derivată și de Hessian (matricea derivatelor parțiale de ordinul 2 ale funcției).

```
import numpy as np
import random
import time

# definirea functiei si a derivatelor
# facem schimbare de baza la logartim
ln2 = np.log(2)

def f(t):
    return np.log(2**t[0] + 2**t[1])

# prima derivata
def gradient(t):
    t1, t2 = t
    num = 2**t1 + 2**t2
    g1 = (2**t1 * ln2) / num
    g2 = (2**t2 * ln2) / num
    return np.array([g1, g2])

# derivatele partiale de ordin 2
def hessian(t):
    t1, t2 = t
    num = 2**t1 + 2**t2
    h11 = (ln2**2 * 2**t1 * 2**t2) / (num**2)
    h12 = -(ln2**2 * 2**t1 * 2**t2) / (num**2)
    return np.array([[h11, h12], [h12, h11]])

# verificam daca respecta conditiile
def este_valid(t):
    return -0.4 <= t[0] <= 2 and 3.2 <= t[1] <= 10 and (t[0] + t[1]) >= 4.5

# generam o solutie valida
def genereaza_valid():
    while True:
        t = np.array([random.uniform(-0.4, 2), random.uniform(3.2, 10)])
        if t[0] + t[1] >= 4.5: return t

timp_alocat = 2.0
start_time = time.time()

x_best = genereaza_valid() # generam prima solutie si o memoram in f_best
(solutia optima globala)
f_best = f(x_best)

while time.time() - start_time < timp_alocat:
    x = genereaza_valid()

    for _ in range(100): # limita de pasi pentru evitarea buclelor infinite
        g = gradient(x)
        h = hessian(x)
```



```

# pasul newton: delta = - H^-1 * g
try:
    alpha = - np.linalg.solve(h + np.eye(2) * 1e-6, g)
except np.linalg.LinAlgError:
    break

x_nou = x + alpha

# verificam daca noua pozitie este valida si este mai buna
if este_valid(x_nou) and f(x_nou) < f(x):
    x = x_nou
else:
    break

# daca f(x) < f(x*) atunci x* = x
if f(x) < f_best:
    x_best = x
    f_best = f(x)

print(f"t1 = {x_best[0]:.4f}, t2 = {x_best[1]:.4f}")
print(f"Emisii minime = {f_best:.4f}")

```

În urma rulării algoritmului de 5 ori s-au obținut următoarele valori:

Nr. rulare	t1 °C	t2 °C	t1 + t2 °C	ln(2 ^{t1} + 2 ^{t2})
1	1.2905	3.2112	4.5017	2.4602
2	1.3035	3.2004	4.5039	2.4562
3	1.2986	3.2035	4.5021	2.4572
4	1.2914	3.2171	4.5085	2.4636
5	1.3022	3.2012	4.5034	2.4564
Media emisiilor nocive				2.45872

Implementarea metodei Simulated Annealing

Pentru problema curentă s-a ales implementarea metodei Simulated Annealing și nu Tabu Search din următoarele motive:

- Tabu Search explorează spațiul memorând într-o listă (așa numita listă Tabu) istoria soluțiilor-candidat recente pentru a refuza revizitarea acestora un timp. Cum într-un spațiu de temperaturi continue există o infinitate de valori între 3.2 și 3.3 de exemplu, ar face ca lista tabu să fie greu de gestionat.
- În Tabu Search se evaluează toți vecinii și se alege cel mai bun, dar într-un spațiu continuu vecinătatea este infinită, motiv pentru care Simulated Annealing este mai potrivită, deoarece alegem un singur vecin aleatoriu.

```

import random
import math

# functia obiectiv
def objective_function(t1, t2):

```

```

    #  $\ln(2^{t1} + 2^{t2})$ 
    return math.log(2**t1 + 2**t2)

# restrictiile
def is_feasible(t1, t2):
    c1 = -0.4 <= t1 <= 2
    c2 = 3.2 <= t2 <= 10
    c3 = (t1 + t2) >= 4.5
    return c1 and c2 and c3

# parametrul Simulated Annealing
temp = 1.0          # temperatura initiala
cooling_rate = 0.995 # rata de racire
min_temp = 0.0001   # pragul de oprire
step_size = 0.1     # pasul de explorare
iterations = 0

# solutia initiala (aceasta trebuie sa fie fezabila)
t1, t2 = 1.0, 4.0
while not is_feasible(t1, t2):
    t1 = random.uniform(-0.4, 2)
    t2 = random.uniform(3.2, 10)

current_sol = {"t1": t1, "t2": t2}
current_fitness = objective_function(current_sol["t1"], current_sol["t2"])

best_sol = current_sol.copy()
best_fitness = current_fitness

# bucla principala
while temp > min_temp:
    iterations += 1

    new_sol = current_sol.copy()
    var_to_change = random.choice(["t1", "t2"])

    new_sol[var_to_change] += random.uniform(-step_size, step_size)

    # verificam daca noua solutie respecta restrictiile
    if is_feasible(new_sol["t1"], new_sol["t2"]):
        new_fitness = objective_function(new_sol["t1"], new_sol["t2"])

        delta_e = new_fitness - current_fitness

        if delta_e < 0 or random.random() < math.exp(-delta_e / temp):
            current_sol = new_sol
            current_fitness = new_fitness

            if current_fitness < best_fitness:
                best_sol = current_sol.copy()
                best_fitness = current_fitness

    temp *= cooling_rate

print(f"--- Rezultat Simulated Annealing ({iterations} iteratii) ---")
print(f"solutie: t1 = {best_sol['t1']:.4f}, t2 = {best_sol['t2']:.4f}")
print(f"nivel minim emisii: {best_fitness:.6f}")

```

În urma rulării algoritmului de 5 ori s-au obținut următoarele valori:

Nr. rulare	t1 °C	t2 °C	t1 + t2 °C	ln(2 ^{t1} + 2 ^{t2})
1	1.2997	3.2005	4.5002	2.455698
2	1.2992	3.2016	4.5008	2.456204
3	1.3001	3.2000	4.5001	2.455506
4	1.3021	3.2000	4.5021	2.455789
5	1.3034	3.2007	4.5041	2.456333
Media emisiilor nocive				2.455905

Analiza comparativă a rezultatelor și a metodelor folosite

În tabelul de mai jos vor fi comparate cele mai bune rezultate obținute de fiecare metodă pentru funcția obiectiv, adică pentru nivelul de emisii nocive și valorile variabilelor t1 și t2.

Metoda de optimizare	t1 °C	t2 °C	t1 + t2 °C	ln(2 ^{t1} + 2 ^{t2})
PySCIPOpt	~1.2999	~3.1999	~4.5000	2.45546
Metoda Newton (cu repornire)	1.3035	3.2004	4.5039	2.4562
Simulated Annealing	1.3001	3.2000	4.5001	2.4555

Comparația performanțelor pe metode

- **PySCIPOpt** (solver exact) este metoda cea mai precisă, identificând punctul de optim matematic exact la granița restricției $t1 + t2 \geq 4.5$
- **Metoda Newton cu repornire** variază între rulări (de la 2.4562 la 2.4636), variație ce este datorată punctelor de start aleatorii. Deși folosește Hessianul, în acest caz a obținut rezultate mai slabe decât Simulated Annealing.
- **Simulated Annealing** a obținut rezultate foarte apropiate de solver (ex. 2.4555 vs 2.4554)

Concluzii

În urma analizei, se pot trage următoarele concluzii:

- Toate cele 3 metode tind să dea rezultate ce sunt apropiate de $t1 \sim 1.3$ și $t2 \sim 3.2$ confirmând faptul că modelul matematic este corect implementat și că optimul global a fost identificat.

- Optimul se află exact pe granița restricției $t_1 + t_2 \geq 4.5$. Deoarece funcția obiectiv este strict crescătoare, minimul va fi întotdeauna cea mai mică valoare permisă de restricții.